

Q1 Create a REST API with the server

→ 1. Set Up AWS and IAM User:

- Create AWS Account: Sign up for an AWS account and create an IAM user with Administrator Access. Configure the AWS CLI with the access keys for authentication.

2. Install Tools

- Install Node.js and the Serverless framework globally. The serverless framework simplifies Lambda and API Gateway development.

3. Create a Serverless Project

- Create a new Serverless project using the AWS Node.js template. This generates the necessary files like `serverless.yml` for configuration and `handler.js` for defining Lambda functions.

4. Define Lambda Functions:

- In the handler file, write the Lambda functions to handle API requests such as fetching or creating user data. These small functions will be triggered by HTTP requests routed through API Gateway.

5. Configure API Gateway

- In `serverless.yml`, specify the events (like GET or POST) that will trigger the Lambda functions. This setup links API Gateway routes to your Lambda functions definitions like user to different operations.

6. Deploy to AWS

- Deploy the project using the Serverless Framework. It will automatically create and configure AWS resources such as Lambda functions, API Gateway, and permissions.

ZTest and Monitor

- Use the provided API Gateway URL to test your API.

AWS CloudWatch logs help monitor and debug any issue in the Lambda function or API Gateway request.

Through this experiment I learned to create and deploy a serverless REST API using AWS Lambda and the Serverless Framework configuration API Gateway, writing Lambda functions and managing cloud resources efficiently with scalable serverless architecture.

Q2 Case Study: Using SonarQube and SonarCloud for Code Quality Analysis

1. Introduction

In this case study we set up SonarQube and SonarCloud to evaluate the quality of various projects including Python, Node.js and Java ensuring high standard by detecting code smells, vulnerabilities and bugs.

2. Creating a SonarQube Profile

To begin SonarQube was installed on a local machine accessed via "http://localhost:9000", and configured by logging in as an admin. A new quality profile was created tailored specifically for Python projects. The profile focused on detecting common issues such as code duplicate maintainability concerns and security vulnerabilities.

3. Analyzing a Python Project with SonarQube

A large Python project was analyzed using the SonarQube scanner which was configured as follows:

```
sonar-scanner -Dsonar.projectKey=python-project -Dsonar.source=./src
```

The analysis report highlighted 15 critical bugs and several code smells, many real related to poor code formatting unused variables and security vulnerabilities. SonarQube provided recommendation to address these issues leading to improved code maintainability and security.

4. Analyzing a Node.js Project

For a Node.js project the SonarQube scanner was integrated via the sonar-project.properties file. The scan revealed 12 vulnerabilities and 20 code smells with issues related to error handling and the use of deprecated functions. Recommendation from SonarQube led to restructuring the code to enhance performance and security.

5. SonarCloud Integration for Java Projects

A Java project hosted on Github was analyzed using SonarCloud. The repository was linked with SonarCloud and SonarLint was installed in VScode for realtime feedback during development. Continuous integration with Github Actions enabled ongoing monitoring of code quality as new commits were made. The analysis identified 10 major bugs along with minor issues,

contributing to more robust and secure code

6. Conclusion

The use of SonarQube and SonarCloud enhanced the code quality of Python, Node.js and Java projects by identifying and resolving critical issues. These tools provided actionable feedback improving maintainability and overall code robustness.

Q3 Terraform 'self-service' Infrastructure Model

- 1) Terraform Modules for self-service infrastructure
- Create Terraform modules that codify the standards for deploying common resources like VPCs, EC2 instances and S3 buckets

Example module for an EC2 instance:

ec2-module/main.tf:

```
variable "instance-type" {  
  default = "t2.micro"
```

```
}
```

```
resource "aws_instance" "example" {
```

```
  ami = ami-12345678
```

```
  instance_type = var.instance-type
```

```
  tags = {
```

```
    Name = "example-instance"
```

```
  }
```

```
}
```

ec2-module/outputs.tf:

```
output "instance-id" {
```

```
} value = aws_instance.example.id
```

Teams can now use this module to deploy EC2 instance with

```
module "ec2" {  
  source = ". / ec2 - module"  
  instance-type = "t2.medium"  
}
```

2) Terraform Cloud Integration with Service Now

- You can integrate Terraform Cloud with Service Now to automate the infrastructure request process.
- Use Terraform API-driven approach Service Now can trigger Terraform runs based on ticket approvals automating resource deployment

Example workflow

- 1) A product team submits a request in Service Now for new infrastructure
- 2) The request triggers a Terraform Cloud updates the service now ticket with the status and resource details
- 3) Creating Terraform Modules for team, Define reusable modules for commonly requested resources like:
 - 1) Networking (VPC, Subnets)
 - 2) Compute (EC2, Autoscaling Groups)
 - 3) Storage (S3, RDS)
 - 4) IAM Roles/Policies

By doing this teams can manage their own infrastructure while maintaining compliance with organizational standard.

